

📘 10차시 SFT + QLoRA 심화 교육자료

> 60분 · 한국어 · Ubuntu/Python 3.11/VS Code · 공식 문서 확인일 2026-11-05

1. 10차시 | SFT + QLoRA

60분 실습형 심화 교안 · Ubuntu · Python 3.11 · VS Code

교육 목표: 09차시의 SFT+LoRA를 4비트 NF4 기본 모델로 확장하고, 기본 가중치와 학습 어댑터의 역할을 구분한다.

진행 순서: 0~10분 복습/개념, 10~20분 원리/구조, 20~30분 환경 확인, 30~45분 코드 실행, 45~55분 결과 점검, 55~60분 평가.

실습 모델: Qwen/Qwen2.5-0.5B-Instruct. 이는 제작자가 선택한 경량 교육 모델이며 원본 과정에서 특정 모델을 강제한 것은 아니다.

선행 자료: 07차시 데이터 전처리, 08차시 SFTTrainer, 09차시 LoRA 어댑터. 이 차시의 학습 데이터는 09차시 검증 데이터를 복사해 사용한다.

완료 조건: --dry-run 검증 통과, QLoRA 코드의 구성요소 설명, 실제 학습 미실행과 실험 결과의 차이를 정확히 표시한다.

2. 배경 및 선수지식

전체 파인튜닝은 기본 모델의 많은 가중치를 직접 갱신하므로 가중치, 기울기, 옵티마이저 상태를 저장할 메모리가 필요하다.

LoRA(Low-Rank Adaptation, 로라·저랭크 적응)는 기존 가중치를 동결하고 작은 저랭크 행렬만 학습한다. 09차시는 일반 정밀도 기본 모델에 LoRA를 적용했다.

QLoRA(Quantized LoRA, 큐로라·양자화 기반 저랭크 적응)는 기본 모델을 4비트로 저장하면서 LoRA 행렬은 학습 가능한 정밀도로 유지한다.

양자화는 저장 공간을 줄이는 표현 기술이다. 4비트로 저장했다고 전체 훈련 메모리가 4비트 수준으로 감소하는 것은 아니다. 활성화값, 연산용 버퍼, 어댑터, 옵티마이저 메모리가 별도로 필요하다.

선수지식: JSONL 대화 데이터, CUDA GPU 확인, BF16/FP16 개념, SFTConfig, LoraConfig, 학습·검증 분리.

안전 기준: 모델과 데이터 라이선스를 확인하고 비공개 학습 데이터·토큰을 GitHub나 공개 로그에 기록하지 않는다.

3. 핵심 원리: NF4와 이중 양자화

NF4(NormalFloat4, 엔에프포·정규분포에 맞춘 4비트 표현)는 정규분포 형태의 가중치에 맞춰 설계된 4비트 양자화 형식이다.

BitsAndBytesConfig(load_in_4bit=True)는 4비트 로딩을 요청한다. bnb_4bit_quant_type="nf4"는 NF4를 선택한다.

bnb_4bit_use_double_quant=True는 양자화 상수도 다시 양자화하여 메모리를 추가 절감하는 옵션이다.

bnb_4bit_compute_dtype는 계산에 사용하는 dtype으로, 4비트 저장 dtype과 다르다. GPU가 BF16을 지원하면 BF16, 아니면 FP16을 사용한다.

메모리 이론 비교: 가중치 5억 개만 가정할 때 FP16 저장 약 1.0GB, 4비트 원시 비트량 약 0.25GB. 이는 메타데이터와 기타 메모리를 제외한 단순 산술 예시다.

QLoRA는 양자화된 기본 가중치를 직접 학습하지 않는다. 역전파는 LoRA 어댑터의 학습 가능한 매개변수로 전달된다.

4. 구조도 및 신호 흐름

아래 구조는 실제 코드의 객체 이름과 대응한다.

JSONL train/validation → load_rows() → Dataset.from_list() → SFTTrainer

MODEL_ID → SFTTrainer(model=MODEL_ID, quantization_config=quant, peft_config=lora)

LoraConfig(r=8, target_modules=[q_proj, v_proj]) → SFTTrainer(peft_config=lora) → trainer.train() → trainer.save_model()

핵심 분리: 4비트 기본 가중치는 동결 상태로 유지되고, LoRA A/B 행렬만 업데이트된다. 모델 저장 폴더는 기본 모델 전체 복사본이 아닌 어댑터 중심이다.

주의: 4비트 양자화 모델을 그대로 일반 FP16 모델로 병합·배포할 수 있다고 가정하지 않는다. 11차시에서 비양자화 기본 모델 재로딩과 병합을 따로 다룬다.

5. Ubuntu 환경 준비 및 설치

작업 폴더: 10_SFT_QLoRA; 실습 코드는 lesson10_sft_qlora.py; 데이터는 data/train.jsonl 및 data/validation.jsonl.

python3.11 -m venv .venv → Python 3.11 가상환경 생성

source .venv/bin/activate → 가상환경 활성화

python -m pip install --upgrade pip → 패키지 설치 도구 갱신

python -m pip install torch transformers datasets accelerate peft trl bitsandbytes → 공식 라이브러리 설치

python -c "import torch; print(torch.cuda.is_available())" → CUDA 사용 가능 여부 확인

python -m pip show torch transformers trl peft bitsandbytes → 실제 설치 버전 기록

주의: CUDA 지원 PyTorch 설치 명령은 GPU 드라이버와 CUDA 환경에 따라 달라진다. 공식 PyTorch 설치 선택기를 확인한다.

6. 실습 코드: 데이터 검사와 4비트 로딩

전체 코드는 동봉된 lesson10_sft_qlora.py에 포함한다. 다음은 핵심 발췌이며 원본 파일을 직접 실행한다.

python lesson10_sft_qlora.py --dry-run → 데이터만 검사하고 GPU·모델 다운로드는 건너뛴다.

python lesson10_sft_qlora.py → GPU 환경에서 모델 다운로드, 4비트 로딩, 5스텝 학습을 시도한다.

핵심 코드: quant = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=compute_dtype)

핵심 코드: trainer = SFTTrainer(model=MODEL_ID, quantization_config=quant, peft_config=lora, ...)

핵심 코드: SFTTrainer에 모델 ID와 양자화 설정을 함께 전달한다.

실제 GPU 학습은 이 자료 제작 환경에서 수행하지 않았다. 성공 로그나 학습 손실 수치를 임의로 제시하지 않는다.

7. 코드 해설 및 검증 기준

load_rows()는 messages 배열과 role/content 형식을 검사한다. 동일한 대화가 학습/검증에 중복되면 오류를 발생시킨다.

compute_dtype는 CUDA BF16 지원 여부를 검사한다. BF16 미지원 GPU에서는 FP16을 선택한다.

현재 코드에서는 TRL의 quantization_config 인자로 모델 4비트 적재 및 PEFT 연동을 수행하며, 모델 수동 래핑을 중복하지 않는다.

LoraConfig의 r=8, lora_alpha=16은 09차시와 같아 비교 시 설정 차이를 줄인다. target_modules는 q_proj, v_proj다.

SFTConfig의 max_steps=5는 교육용 짧은 동작 점검이다. 품질 향상이나 수렴을 입증하기에 충분한 학습량이 아니다.

검증 결과와 학습 결과를 구분한다: Python 구문 검사 PASS / 데이터 DRY RUN PASS / CUDA 학습 미검증 / 품질 개선 미검증.

검증 손실이 낮아졌다고 실제 응답 품질이 자동으로 좋아진다고 단정하지 않는다. 12차시에서 독립 평가한다.

8. 따라하기 실습: 60분 운영

① 0~10분: 09차시 LoRA와 QLoRA의 차이를 문장으로 설명한다. 기본 가중치 동결과 4비트 저장을 구분한다.

② 10~20분: 코드에서 quant, model, lora, config, trainer 순으로 찾아 데이터 흐름도에 연결한다.

③ 20~30분: 가상환경 활성화, 설치 버전과 CUDA 인식 확인. GPU가 없으면 --dry-run만 수행한다.

④ 30~35분: python -m py_compile lesson10_sft_qlora.py 로 문법 확인.

⑤ 35~40분: python lesson10_sft_qlora.py --dry-run 실행. 예상: DATA PASS: train=10, validation=2, overlap=0.

⑥ 40~50분: CUDA GPU가 있는 경우 python lesson10_sft_qlora.py 실행 후 어댑터 저장 여부 확인. GPU 미보유자는 구조도·설정값 검토.

⑦ 50~55분: 09차시와 설정 비교표 작성: 기본 가중치 정밀도, LoRA 랭크, 학습 스텝, 배치, 데이터.

⑧ 55~60분: 성공/실패/미실행을 구분하여 실습 기록을 작성한다.

9. 오류 대처와 확장 실습

CUDA GPU가 감지되지 않음: nvidia-smi, PyTorch CUDA 빌드, 드라이버와 GPU 호환성 확인. CPU에서 4비트 QLoRA 학습을 강행하지 않는다.

bitsandbytes 로딩 오류: 설치 버전과 플랫폼 지원을 공식 문서로 확인한다. 패키지 이름만 같아도 백엔드 지원은 다를 수 있다.

Out of memory: max_length와 배치 크기 축소, gradient_checkpointing 확인, 다른 GPU 작업 종료. 단순히 4비트라서 메모리가 항상 충분한 것은 아니다.

토크나이저 패딩 오류: pad_token이 없으면 eos_token으로 설정한다. 실제 패딩 정책과 손실 마스킹도 점검한다.

데이터 형식 오류: messages 내 role/content 문자열을 확인한다. 학습/검증 중복은 완전 동일 대화 기준만 검사하므로 의미적 누출은 별도 검토한다.

확장 실습 A: bnb_4bit_use_double_quant 옵션의 이론적 효과를 설명한다. 실측값은 GPU 환경에서만 기록한다.

확장 실습 B: 09차시와 10차시의 GPU 최대 할당 메모리를 동일 조건에서 비교한다. 측정 장비·버전·배치 조건을 함께 남긴다.

10. 평가·체크리스트·참고자료

평가 1: QLoRA에서 기본 모델 가중치와 LoRA 어댑터 중 어느 쪽을 학습하는가? → LoRA 어댑터.

평가 2: NF4 저장 정밀도와 bnb_4bit_compute_dtype의 관계는? → 저장과 계산의 자료형은 다르다.

평가 3: --dry-run 성공으로 GPU 학습 성공을 주장할 수 있는가? → 불가.

체크리스트: [ ] 데이터 구조 검사 [ ] 09차시와 동일 데이터 [ ] NF4 설정 [ ] LoRA 설정 [ ] 모델 저장 경로 [ ] GPU 학습 실행 여부 기록.

참고: Hugging Face TRL SFTTrainer https://huggingface.co/docs/trl/sft_trainer

참고: Hugging Face Transformers bitsandbytes https://huggingface.co/docs/transformers/quantization/bitsandbytes

참고: Hugging Face PEFT k-bit training https://huggingface.co/docs/peft/developer_guides/quantization

참고: PyTorch 설치 https://pytorch.org/get-started/locally/

공식 문서 확인일: 2026-11-05. 실제 라이브러리 버전별 GPU 실행 호환성은 미검증. 다음 11차시는 병합·GGUF·Ollama 배포다.

전체 실습 코드

```
"""10차시: 4비트 NF4 + SFTTrainer + LoRA; --dry-run은 GPU 없이 데이터 검사."""
import argparse
import json
from pathlib import Path

MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"  # 제작상 선택한 교육용 모델


def load_rows(path):
    """대화형 JSONL 데이터의 기본 구조를 검증한다."""
    rows = []
    for n, line in enumerate(Path(path).read_text(encoding="utf-8").splitlines(), 1):
        if not line.strip():
            continue
        row = json.loads(line)
        msgs = row.get("messages")
        if not isinstance(msgs, list) or not msgs:
            raise ValueError(f"{path}:{n}: messages 배열 누락")
        roles = []
        for msg in msgs:
            if not isinstance(msg, dict) or msg.get("role") not in {"system", "user", "assistant"}:
                raise ValueError(f"{path}:{n}: role 오류")
            if not isinstance(msg.get("content"), str) or not msg["content"].strip():
                raise ValueError(f"{path}:{n}: content 오류")
            roles.append(msg["role"])
        if "user" not in roles or "assistant" not in roles:
            raise ValueError(f"{path}:{n}: user와 assistant가 모두 필요")
        rows.append({"messages": msgs})
    if not rows:
        raise ValueError(f"{path}: 빈 데이터")
    return rows


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--train", default="data/train.jsonl")
    parser.add_argument("--validation", default="data/validation.jsonl")
    parser.add_argument("--output", default="outputs/lesson10_qlora")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    train, valid = load_rows(args.train), load_rows(args.validation)
    canonical = lambda x: json.dumps(x["messages"], ensure_ascii=False, sort_keys=True)
    if {canonical(x) for x in train} & {canonical(x) for x in valid}:
        raise ValueError("학습 · 검증 데이터의 완전 동일 대화 중복")
    print(f"DATA PASS: train={len(train)}, validation={len(valid)}, overlap=0")
    if args.dry_run:
        print("DRY RUN PASS: 모델 다운로드 · 4비트 변환 · GPU 학습 미실행")
        return

    import torch
    from datasets import Dataset
    from transformers import AutoTokenizer, BitsAndBytesConfig
    from peft import LoraConfig
    from trl import SFTConfig, SFTTrainer

    if not torch.cuda.is_available():
        raise RuntimeError("CUDA GPU 필요: 먼저 --dry-run으로 데이터 검사")
```

```
    # BF16 지원 GPU이면 BF16, 아니면 FP16으로 연산한다.
    compute_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    quant = BitsAndBytesConfig(
        load_in_4bit=True, bnb_4bit_quant_type="nf4",
        bnb_4bit_use_double_quant=True,
        bnb_4bit_compute_dtype=compute_dtype,
    )
    # (변경) TRL에 모델 ID와 양자화 설정을 직접 전달한다.
    # TRL/PEFT가 기본 가중치의 4비트 적재와 LoRA 학습 준비를 담당한다.
    lora = LoraConfig(
        r=8, lora_alpha=16, lora_dropout=0.05,
        target_modules=["q_proj", "v_proj"], bias="none", task_type="CAUSAL_LM",
    )
    config = SFTConfig(
        output_dir=args.output, max_length=256,
        per_device_train_batch_size=1, gradient_accumulation_steps=2,
        max_steps=5, learning_rate=1e-4,
        logging_steps=1, eval_strategy="steps", eval_steps=5,
        save_strategy="no", report_to="none", seed=42,
        bf16=(compute_dtype == torch.bfloat16), fp16=(compute_dtype == torch.float16),
        gradient_checkpointing=True, assistant_only_loss=False,
    )
    trainer = SFTTrainer(
        model=MODEL_ID, args=config, train_dataset=Dataset.from_list(train),
        eval_dataset=Dataset.from_list(valid), processing_class=tokenizer,
        peft_config=lora, quantization_config=quant,
    )
    trainer.model.print_trainable_parameters()
    result = trainer.train()
    print("학습 손실:", result.training_loss)
    print("검증 결과:", trainer.evaluate())
    # 저장 대상은 기본 모델이 아닌 학습된 LoRA 어댑터이다.
    trainer.save_model(args.output)
    tokenizer.save_pretrained(args.output)
    print("QLoRA 어댑터 저장:", args.output)


if __name__ == "__main__":
    main()
```

실습 데이터

`data/train.jsonl`, `data/validation.jsonl`은 09차시 학습 데이터와 동일한 파일을 재사용한다.

적대적 검증·개선 기록 (2026-11-05)

반영: 공식 TRL 최신 SFTTrainer의 quantization_config 경로로 정리하고, 07차시 10/2 학습·검증 데이터를 연결했습니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)